JavaScript Deep Fundamentals for Interviews (Hoisting, Closures, Scope, Async, Promises)

This file covers what interviewers expect senior JS engineers to articulate clearly.

1) Hoisting and Temporal Dead Zone (TDZ)

1.1 var Hoisting

```
console.log(x); // undefined (NOT error)
var x = 5;
console.log(x); // 5
```

Why:

- var declarations are hoisted to top and initialized as `undefined`.
- Function hoisting: entire function moved to top, can call before declaration.

1.2 let/const TDZ

```
console.log(y); // ReferenceError: Cannot access 'y' before initialization
let y = 10;
```

Why:

- let/const are hoisted but NOT initialized.
- Code from declaration start to initialization is "temporal dead zone".
- Accessing throws ReferenceError.

1.3 Hoisting Interview Question

```
function test() {
  console.log(typeof foo); // "undefined"
  console.log(foo); // ReferenceError (if we tried)
  var foo = 1;
  console.log(foo); // 1
}
test();
```

Expected: "undefined" first because `var` is hoisted and initialized to `undefined`.

2) Scope and Scope Chain

2.1 Global, Function, Block Scope

```
var global = "g";

function f() {
```

```
  var functionLocal = "f";

  if (true) {
    let blockLocal = "b";
    var weirdVar = "weird"; // function-scoped, not block-scoped
  }

  console.log(weirdVar); // "weird" (accessible here)
  console.log(blockLocal); // ReferenceError
}

f();
```

Interview expectation:

- var is function-scoped.
- let/const are block-scoped.

2.2 Scope Chain Lookup

```
var a = 1;

function outer() {
  var b = 2;

  function inner() {
    var c = 3;
    console.log(a, b, c); // 1, 2, 3
  }

  inner();
}

outer();
```

Lookup chain: inner -> outer -> global.

3) Closures (Most Misunderstood Topic)

3.1 Closure Definition

A closure is a function that has access to variables from its enclosing scope even after the enclosing function has returned.

```
function makeCounter() {
  let count = 0;
  return function increment() {
    return ++count;
  };
}
```

```
const counter = makeCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
```

Why:

- `increment` closes over `count`.
- `count` persists in memory even after `makeCounter` returns.

3.2 Closure Pitfall: Loop with var

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// Output: 3, 3, 3 (because i is function-scoped and shared)
```

Fix:

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// Output: 0, 1, 2 (each iteration gets its own i)
```

3.3 Closure Advanced: Module Pattern

```
const module = (() => {
  let private = "secret";

  return {
    getPrivate() {
      return private;
    },
    setPrivate(val) {
      private = val;
    }
  };
})();

console.log(module.getPrivate()); // "secret"
console.log(module.private); // undefined (truly private)
```

Interview explanation:

- IIFE + closure creates private state.
- Classic pattern before ES6 classes.

4) Prototype and Prototype Chain

4.1 Prototype Basics

```
function Animal(name) {
  this.name = name;
}

Animal.prototype.speak = function() {
  console.log(`${this.name} makes a sound`);
};

const dog = new Animal("Dog");
dog.speak(); // "Dog makes a sound"
```

Property lookup:

1. own properties,
2. prototype,
3. prototype's prototype,
4. null (chain ends).

4.2 Prototype Chain Inspection

```
console.log(dog.hasOwnProperty("name")); // true
console.log(dog.hasOwnProperty("speak")); // false
console.log("speak" in dog); // true
```

4.3 Avoid Prototype Mutation in Loops

```
for (let key in obj) {
  if (obj.hasOwnProperty(key)) {
    // safe: only own properties
  }
}
```

Interview expectation:

- know the difference between own and inherited properties.

5) this Binding (Context and Explicit Binding)

5.1 Implicit Binding

```
const obj = {
  name: "obj",
  greet: function() {
    console.log(this.name);
  }
};

obj.greet(); // "obj"
```

```
const fn = obj.greet;
fn(); // undefined (this is global or undefined in strict)
```

Rule: this depends on how function is called.

5.2 Explicit Binding: call, apply, bind

```
function greet(greeting) {
  console.log(`${greeting}, ${this.name}`);
}

const obj = { name: "Alice" };

greet.call(obj, "Hello"); // "Hello, Alice"
greet.apply(obj, ["Hi"]); // "Hi, Alice"

const boundGreet = greet.bind(obj);
boundGreet("Hey"); // "Hey, Alice"
```

5.3 Arrow Functions Don't Have Their Own this

```
const obj = {
  name: "obj",
  normalMethod: function() {
    console.log(this.name); // "obj"
  },
  arrowMethod: () => {
    console.log(this.name); // undefined (this is global)
  }
};

obj.normalMethod();
obj.arrowMethod();
```

Interview implication:

- arrow functions useful in callbacks, bad for object methods.

6) Promises and Async/Await (Deep Dive)

6.1 Promise States and Microtask Execution

```
const p = new Promise((resolve, reject) => {
  console.log("1. executor runs immediately");
  resolve("value");
  console.log("2. after resolve");
});

console.log("3. after new Promise");
```

```
p.then(val => console.log("4. then callback:", val));

console.log("5. last sync");
```

Expected output:

- 1. executor runs immediately
- 2. after resolve
- 3. after new Promise
- 5. last sync
- 4. then callback: value

Why:

- executor runs synchronously.
- `.then` callback queued as microtask, runs after all sync code.

6.2 Promise Chaining and Error Handling

```
Promise.resolve(1)
  .then(x => x + 1) // 2
  .then(x => {
    throw new Error("oops");
  })
  .then(x => console.log("skipped", x))
  .catch(e => console.log("caught:", e.message))
  .then(x => console.log("after catch")); // runs
```

Output:

- caught: oops
- after catch

Rule:

- rejected promise skips `.then`, goes to `.catch`.
- `.catch` returns resolved promise (unless it throws).

6.3 Promise.all vs Promise.allSettled vs Promise.race

```
// Promise.all: rejects if ANY rejects, resolves when all resolve
Promise.all([p1, p2, p3]);

// Promise.allSettled: always resolves with array of { status, value|reason }
Promise.allSettled([p1, p2, p3]);

// Promise.race: resolves/rejects when FIRST settles
Promise.race([p1, p2, p3]);
```

6.4 Async/Await (Syntactic Sugar Over Promises)

```
async function fetch Data() {
  try {
    const data = await fetchUser(); // pause until promise resolves
    console.log(data);
    return data;
  } catch (e) {
    console.log("error:", e);
  }
}

// equivalent to:
function fetchData() {
  return fetchUser().then(data => {
    console.log(data);
    return data;
  }).catch(e => console.log("error:", e));
}
```

6.5 Await in Loops

Wrong (slow, sequential):

```
for (const id of ids) {
  await fetch(`/api/${id}`); // waits for each
}
```

Better (parallel):

```
await Promise.all(ids.map(id => fetch(`/api/${id}`)));
```

7) Tricky Output Questions: Advanced

Q1: Closure + Async

```
const funcs = [];
for (var i = 0; i < 3; i++) {
  funcs.push(() => {
    console.log(i);
  });
}
funcs.forEach(f => f()); // 3, 3, 3
```

Fix with let:

```
for (let i = 0; i < 3; i++) {
  funcs.push(() => console.log(i));
```

```
}
funcs.forEach(f => f()); // 0, 1, 2
```

Q2: Promise Chaining

```
Promise.resolve(1)
  .then(x => Promise.resolve(x + 1))
  .then(x => {
    throw x;
  })
  .then(x => console.log("1:", x))
  .catch(x => console.log("2:", x))
  .then(x => console.log("3:", x));
```

Output:

- 2: 2
- 3: undefined

Q3: setTimeout in Promise

```
Promise.resolve()
  .then(() => {
    console.log("A");
    setTimeout(() => console.log("B"), 0);
  })
  .then(() => console.log("C"));

console.log("D");
```

Output:

- D
- A
- C
- B

8) Type Coercion (Tricky Comparisons)

```
console.log(0 == false); // true (coercion)
console.log(0 === false); // false (no coercion)

console.log("" == 0); // true
console.log("" === 0); // false

console.log(null == undefined); // true
console.log(null === undefined); // false
```

Interview guidance:

- always use `===` unless coercion is intentional.

9) Advanced Patterns: Debounce, Throttle, Memoization

9.1 Debounce (Closure + Timer)

```
function debounce(fn, delay) {
  let timeout;
  return function(...args) {
    clearTimeout(timeout);
    timeout = setTimeout(() => fn(...args), delay);
  };
}

const search = debounce((query) => {
  console.log("searching:", query);
}, 300);

search("a"); search("ab"); search("abc"); // only "abc" fires
```

9.2 Throttle (Closure + Time Check)

```
function throttle(fn, delay) {
  let lastRun = 0;
  return function(...args) {
    const now = Date.now();
    if (now - lastRun > delay) {
      fn(...args);
      lastRun = now;
    }
  };
}

const scroll = throttle(() => console.log("scrolling"), 100);
// scroll runs max once per 100ms
```

9.3 Memoization (Closure + Cache)

```
function memoize(fn) {
  const cache = {};
  return function(...args) {
    const key = JSON.stringify(args);
    if (key in cache) return cache[key];
    const result = fn(...args);
    cache[key] = result;
    return result;
  };
}
```

```
const expensiveFn = memoize((n) => {
  console.log("computing...");
  return n * 2;
});

console.log(expensiveFn(5)); // computing... 10
console.log(expensiveFn(5)); // 10 (cached, no log)
```

10) WeakMap, WeakSet, and Memory Leaks

10.1 WeakMap for Private State

```
const privateData = new WeakMap();

class User {
  constructor(id) {
    privateData.set(this, { id, password: "secret" });
  }

  verifyPassword(pwd) {
    return privateData.get(this).password === pwd;
  }
}

const user = new User(1);
console.log(privateData.get(user)); // { id, password }
console.log(user.password); // undefined (truly private)
```

10.2 WeakMap Garbage Collection

- WeakMap keys are weakly referenced.
- If key is GC'd, entry is removed.
- Prevents memory leaks in long-lived maps.

11) Generators and Iterators

11.1 Generator Basics

```
function* gen() {
  console.log("start");
  yield 1;
  console.log("between");
  yield 2;
  console.log("end");
}

const g = gen();
console.log(g.next()); // { value: 1, done: false }, prints "start"
```

```
console.log(g.next()); // { value: 2, done: false }, prints "between"
console.log(g.next()); // { value: undefined, done: true }, prints "end"
```

Interview use:

- generators useful for lazy evaluation and control flow.

12) Symbol and Symbol.iterator

```
const mySymbol = Symbol("unique");

const obj = {
  [mySymbol]: "hidden value"
};

console.log(obj[mySymbol]); // "hidden value"
console.log(Object.keys(obj)); // [] (symbols not enumerable)

// Symbol.iterator for custom iteration
const iter = {
  [Symbol.iterator]() {
    let i = 0;
    return {
      next: () => ({
        value: i++,
        done: i > 3
      })
    };
  }
};

for (const val of iter) console.log(val); // 0, 1, 2
```

13) Proxy and Reflect

13.1 Proxy for Interception

```
const target = { x: 1, y: 2 };

const handler = {
  get(t, key) {
    console.log(`accessing ${key}`);
    return t[key];
  },
  set(t, key, val) {
    console.log(`setting ${key} to ${val}`);
    t[key] = val;
    return true;
  }
};
```

```
const proxy = new Proxy(target, handler);
console.log(proxy.x); // accessing x, 1
proxy.x = 10; // setting x to 10
```

Interview expectation:

- proxies useful for validation, logging, lazy loading.

14) JavaScript Engine Internals

These topics show up in stronger L4/L5 interview loops because they separate API familiarity from runtime understanding.

14.1 Engine vs Runtime

JavaScript engine examples:

- V8,
- SpiderMonkey,
- JavaScriptCore.

What the engine does:

- parse source,
- build AST,
- generate bytecode or intermediate representation,
- interpret,
- JIT-compile hot paths,
- run garbage collection.

What the runtime provides:

- timers,
- DOM,
- fetch,
- storage,
- event loop integration,
- worker APIs,
- networking and file APIs depending on platform.

Interview line:

- ECMAScript defines the language. The engine executes the language. The runtime provides the host APIs.

14.2 Parse, Interpret, JIT, Optimize

High-level pipeline:

1. Parse source.
2. Produce AST.
3. Generate bytecode / baseline representation.
4. Interpret quickly.
5. Detect hot paths.
6. Optimize hot code with JIT.
7. Deoptimize if assumptions become invalid.

Important interview point:

- Modern engines trade startup speed against peak performance. They usually start with a fast baseline path, then optimize frequently executed code.

14.3 Hidden Classes / Shapes

Engines optimize objects better when property layout is predictable.

Good:

```
function User(id, name) {
  this.id = id;
  this.name = name;
}
```

Riskier:

```
const user = {};
user.id = 1;
if (flag) user.extra = true;
user.name = "Ada";
```

Why interviewers care:

- adding properties in inconsistent order can create many shapes,
- megamorphic call sites are harder to optimize,
- stable object layout usually helps engines.

14.4 Inline Caches

When a property access like `user.name` happens repeatedly, engines cache how that lookup succeeds.

Kinds of call sites:

- monomorphic: one shape seen,
- polymorphic: a few shapes seen,
- megamorphic: many shapes seen.

Senior interview line:

- predictable object shapes and stable property access patterns help inline caches stay effective.

14.5 Deoptimization

Optimized code can be thrown away if engine assumptions stop being true.

Common reasons:

- changing value types unexpectedly,
- highly dynamic object mutation,
- `try/catch` in historically sensitive paths,
- `arguments` and other deopt-prone patterns in some engines,
- extremely polymorphic call sites.

Interview nuance:

- you do not usually micro-optimize application code around deopts first, but understanding them helps explain why highly dynamic code can benchmark poorly.

14.6 Garbage Collection

You should know the basic model:

- objects become collectible when unreachable,
- modern engines use generational GC,
- short-lived objects are common and optimized for,
- old-generation collections are more expensive.

Typical terms:

- mark-and-sweep,
- generational GC,
- young generation,
- old generation,
- stop-the-world pauses,
- incremental or concurrent collection.

Interview line:

- memory leaks in JavaScript are usually logical leaks from retaining references, not forgetting to call free().

15) Browser Runtime and Web Platform Theory

15.1 Call Stack, Web APIs, Queues, and Rendering

Browser execution model at high level:

1. Run synchronous JS on the call stack.
2. Host APIs like timers, DOM events, and fetch are managed outside the engine.
3. Completed callbacks enter task queues.
4. Microtasks run before the next macrotask.
5. Browser may render between tasks when appropriate.

Interview point:

- rendering does not happen in the middle of a long synchronous task. Long JS blocks painting and input responsiveness.

15.2 Microtasks vs Macrotasks

Microtasks:

- `Promise.then`,
- `queueMicrotask`,
- MutationObserver callbacks.

Macrotasks:

- `setTimeout`,
- `setInterval`,
- DOM events,
- postMessage task delivery,
- network/task callbacks depending on host integration.

Senior interview line:

- after a macrotask finishes, the runtime drains the microtask queue before moving to the next macrotask and before the browser gets its next good chance to paint.

15.3 `requestAnimationFrame` vs `setTimeout`

Use `requestAnimationFrame` for visual updates tied to paint.

Why:

- aligned with browser paint cycle,
- avoids unnecessary work in background tabs,
- better for animation smoothness.

Use `setTimeout` for general delayed work, not animation scheduling.

15.4 Event Propagation: Capture, Target, Bubble

Three phases:

1. Capture phase.
2. Target phase.
3. Bubble phase.

Interview expectations:

- know `event.target` vs `event.currentTarget`,
- know event delegation,
- know when `stopPropagation()` changes behavior.

15.5 Event Delegation

Important for performance and dynamic DOM trees.

```
list.addEventListener("click", (event) => {
  if (event.target.matches("button.delete")) {
    removeItem(event.target.dataset.id);
  }
});
```

Why it matters:

- fewer listeners,
- works for dynamically inserted children,
- common in high-volume DOM interaction questions.

16) LRU Cache and Common Grilling Data Structures

LRU cache is a very common frontend or full-stack machine-coding discussion because it combines:

- hash lookup,
- ordering,
- eviction,
- complexity reasoning.

16.1 LRU Definition

LRU means Least Recently Used.

Eviction rule:

- when capacity is full, remove the item that has not been used for the longest time.

16.2 Expected Complexity

Strong interview answer:

- `get` : $O(1)$
- `put` : $O(1)$

Typical implementation:

- Hash map for direct lookup.
- Doubly linked list for recency ordering.

16.3 Simple JavaScript Version with `Map`

In JavaScript interviews, a practical `Map` -based version is often acceptable because insertion order is preserved.

```
class LRUCache {
  constructor(capacity) {
    this.capacity = capacity;
    this.map = new Map();
  }

  get(key) {
    if (!this.map.has(key)) return -1;

    const value = this.map.get(key);
    this.map.delete(key);
    this.map.set(key, value);
    return value;
  }

  put(key, value) {
    if (this.map.has(key)) {
      this.map.delete(key);
    }

    this.map.set(key, value);

    if (this.map.size > this.capacity) {
      const lruKey = this.map.keys().next().value;
      this.map.delete(lruKey);
    }
  }
}
```

16.4 Follow-up Questions on LRU

You should be ready for:

- How would you make it thread-safe in another language?
- What if entries expire by TTL as well as recency?
- How would you implement LFU instead?
- What happens if values are huge and eviction is expensive?
- Would you use LRU for API response caching, image caching, or memoization?

16.5 Browser-Relevant Cache Interview Angles

Interviewers may connect LRU concepts to:

- in-memory client caches,
- React query caches,
- image or asset caching,
- CDN and proxy eviction policies,
- service worker or application-level cache layers.

17) Senior Revision Checklist

- [ ] Explain hoisting for var, let, const.
- [ ] Explain TDZ.
- [ ] Explain scope chain.
- [ ] Explain closure and closure pitfalls.
- [ ] Explain prototype chain.
- [ ] Explain this binding in different contexts.
- [ ] Explain promise states and microtask.
- [ ] Explain async/await vs promises.
- [ ] Explain engine vs runtime.
- [ ] Explain hidden classes, inline caches, and deoptimization at a high level.
- [ ] Explain browser event loop, microtasks vs macrotasks, and paint timing.
- [ ] Explain event delegation and propagation phases.
- [ ] Implement and analyze an LRU cache.
- [ ] Implement debounce, throttle, memoize.
- [ ] Explain WeakMap use cases.
- [ ] Explain Symbol and Symbol.iterator.
- [ ] Explain Proxy use cases.